Experiment 3: Implementation and analysis of Spatial Filtering Techniques using low-pass and High-pass Filters

Name- Mohd Zaid

Course & Branch- B.TECH CSE

CUID- CU24250049

SUBJECT - Computer Vision Lab

SEM & Section - 5th SEM , "B"

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from google.colab.patches import cv2_imshow

uploaded = files.upload()
image_path = list(uploaded.keys())[0]

img_color = cv2.imread(image_path)
img_color = cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img_color, cv2.COLOR_RGB2GRAY)
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
plt.figure(figsize=(6, 6))
plt.imshow(img_gray, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.show()
```




```
gaussian_blur = cv2.GaussianBlur(img_gray, (5, 5), 0)

plt.imshow(gaussian_blur, cmap='gray')
plt.title("Gaussian Blur")
plt.axis('off')
plt.show()
```




```
median_blur = cv2.medianBlur(img_gray, 5)

plt.imshow(median_blur, cmap='gray')
plt.title("Median Filter")
plt.axis('off')
plt.show()
```



```
average_blur = cv2.blur(img_gray, (5, 5))

plt.imshow(average_blur, cmap='gray')
plt.title("Average Filter")
plt.axis('off')
plt.show()
```




```
laplacian = cv2.Laplacian(img_gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

plt.imshow(laplacian, cmap='gray')
plt.title("Laplacian Filter")
plt.axis('off')
plt.show()
```



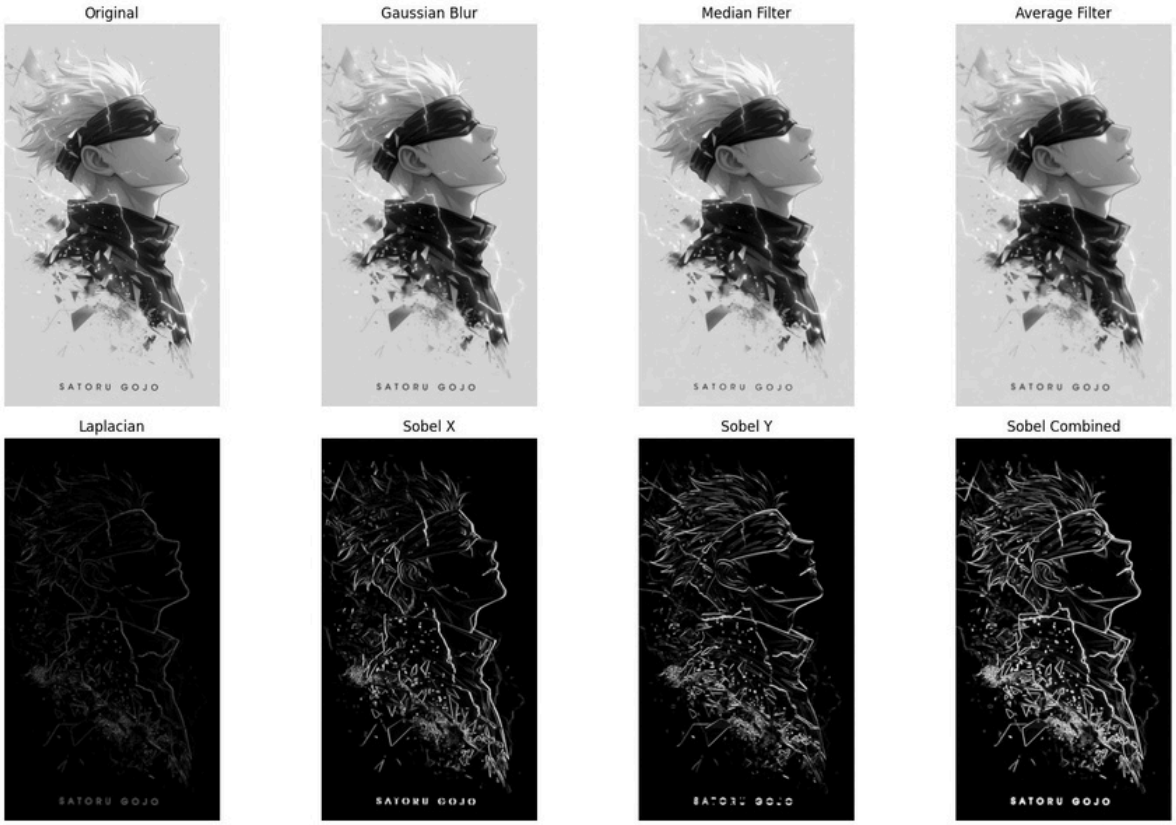

```
sobel_x = cv2.Sobel(img_gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img_gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)
sobel_combined = cv2.bitwise_or(sobel_x, sobel_y)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(sobel_x, cmap='gray'); plt.title("Sobel X")
plt.subplot(1, 3, 2); plt.imshow(sobel_y, cmap='gray'); plt.title("Sobel Y")
plt.subplot(1, 3, 3); plt.imshow(sobel_combined, cmap='gray'); plt.title("So
plt.tight_layout()
plt.show()
```




```
titles = ['Original', 'Gaussian Blur', 'Median Filter', 'Average Filter',
          'Laplacian', 'Sobel X', 'Sobel Y', 'Sobel Combined']
images = [img_gray, gaussian_blur, median_blur, average_blur,
          laplacian, sobel_x, sobel_y, sobel_combined]

plt.figure(figsize=(16, 10))
for i in range(len(images)):
plt.subplot(2, 4, i + 1)
plt.imshow(images[i], cmap='gray')
plt.title(titles[i])
plt.axis('off')
plt.tight_layout()
plt.show()
```




```
def add_salt_pepper_noise(image, amount=0.02):
    noisy = np.copy(image)
    num_salt = np.ceil(amount * image.size * 0.5)
```

```
        num_pepper = np.ceil(amount * image.size * 0.5)

        coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shap
        noisy[coords[0], coords[1]] = 255

        coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.sh
        noisy[coords[0], coords[1]] = 0
        return noisy

noisy_img   =   add_salt_pepper_noise(img_gray)
median_denoised = cv2.medianBlur(noisy_img, 5)
average_denoised = cv2.blur(noisy_img, (5, 5))

plt.figure(figsize=(14, 5))
plt.subplot(1, 3, 1); plt.imshow(noisy_img, cmap='gray'); plt.title("Salt &
plt.subplot(1, 3, 2); plt.imshow(median_denoised, cmap='gray'); plt.title("M
plt.subplot(1, 3, 3); plt.imshow(average_denoised, cmap='gray'); plt.title("
plt.tight_layout()
plt.show()
```



Question Answers

1. What is spatial filtering?

Ans-> Spatial filtering is an image enhancement technique that modifies a pixel's value based on the values of its neighboring pixels, using a small matrix called a kernel (or mask) that slides over the image via convolution. It's used for smoothing,

sharpening, noise removal, and edge detection directly in the pixel domain (as opposed to transforming to frequency domain first).

2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Ans->

- Low-pass filters allow slow variations (smooth regions) to pass through while suppressing high-frequency components like noise and edges — result: blurred/smoothed image. Examples: Gaussian, Average/Mean, Median.
- High-pass filters suppress slow variations and emphasize high-frequency components like edges and fine details — result: sharpened image with enhanced boundaries. Examples: Laplacian, Sobel, Prewitt.

3. Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications

Ans-> Average, Gaussian, and Median filters all smooth images but work differently. The Average Filter replaces each pixel with the plain mean of its neighborhood, causing heavy blurring of edges. The Gaussian Filter uses a weighted average (more weight to center pixels), giving smoother results with better edge preservation. The Median Filter replaces each pixel with the median value of its neighborhood, making it ideal for removing salt-and-pepper noise without blurring edges much, unlike the other two.

4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Ans-> Salt-and-pepper noise appears as extreme outlier pixel values (pure black/white). Since the median is not influenced by outliers the way a mean is, sorting the neighborhood and picking the middle value effectively discards these extreme values while preserving edges and detail better than averaging filters.